EPFL Quantum Hackathon 2026 – AXA x Quobly

Reinsurance Allocation Optimization

Ahmet Alperen Tekin, Mamoun Chami, Valentin Walendy

Problem Definition: Reinsurance Allocation

- Choose among N different coverages to minimize expected risk of large losses.
- Each coverage has an insurance premium and a risk assesment value

Problem Definition: Constrained Binary Optimization

- *x is a binary vector of size (N,1), where N is the number of policies*
- $\min r^T x$, *where r is the risk vector* $(N, 1)$, *with the constraint*
- $c^T x - b < 0$, *where c is the premium vector and b is the budget*

Risk Assessment: Dataset

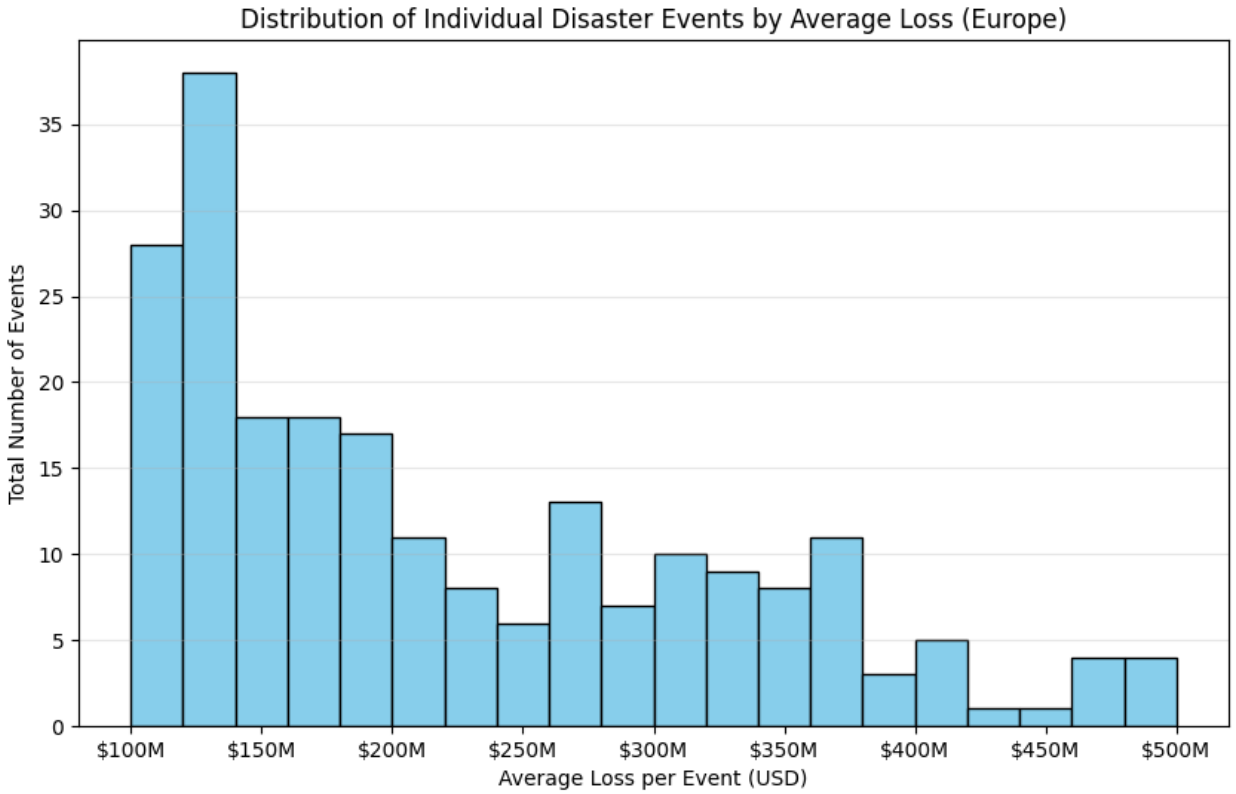
- «Natural Disasters Emergency Events Database»

- Each row represents (relevant):
  - Country, Year, # of Disaster Occurrences, Total Loss (Price Adjusted USD)

- We filter by (relevant):
  - European countries, Weather Type Disasters, 100M < Loss per Event < 500M

Risk Assessment: Histogram

- From the filtered samples, we calculate a histogram of:
  - # of occurrences, price adjusted loss per event between (x, x+dx) $

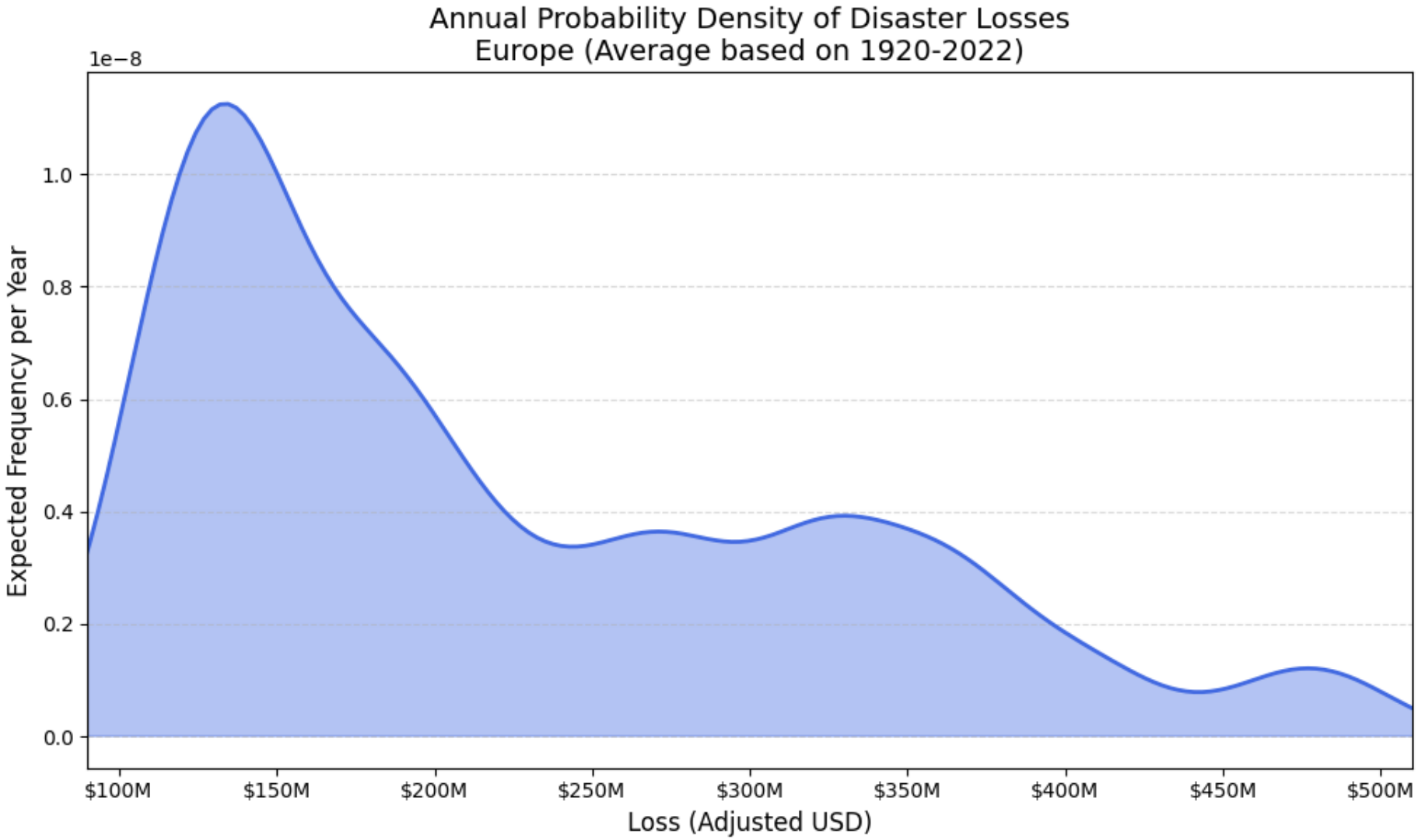


Risk Assessment: Prob. Density Function

- The dataset contains disasters from 1900 to 2022 (103 years)

- From the histogram, we calculate a probability density function, that represents:
  - The probability of an event occurring with loss between (x, x+dx) $

Risk Assessment: Prob. Density Function



Risk Assessment: Prob. Density Function

- Looking at the Probability Density Function (pdf), we calculate that the probability of an event occurring in a given year would be approximately 1.7

- We assume %10 of the events will be insured by us. By dividing the pdf by 10, we get the pdf for the losses to be deducted from our insurance company.

Risk Assessment: Modelling Policies

- The policies are modeled by two parameters, threshold and proportion.

- Below the set threshold, the reinsurer pays for all the damages,
- Above the threshold, the reinsurer pays for a portion of damages, determined by the policy proportion

- Each policy also has a premium, which will be paid for subscription

Risk Assessment: Extracting r_i Values

$$\int_{100M}^{500M} f_i(x)p(x)dx = r_i$$

Where

$f_i(x, threshold, prop)$ is the policy payout function

$\mathrm{p}(x)$ is the probability density function for our insurance company

Risk Assessment: r_i Explanation

- The risk assessment matrix essentially tells us the expected value of the payment to be made by the reinsurer.
- For each policy, we pay an insurance premium, so the problem takes the form:
- $\max(r - c)^T x \, , where\ r\ is\ the\ risk\ vector\ (N, 1), with\ the\ constraint$
- $c^T x\ - b < 0, where\ c\ is\ the\ premium\ vector\ and\ b\ is\ the\ budget$

Problem Formulation: CBO to UBO

- We need to convert the problem from a constrained binary maximization problem to an unconstrained minimization problem.

- In order to make this conversion, we add the contraint as a penalty term. The final UBO takes the form:

$$\min\ -(r-c)^T x + \lambda(c^T x - B)^2$$

Why Unconstrained Binary Optimization

- Unconstrained Binary Optimization Probem is advantageous, because we can run:
  - Variational Quantum Eigensolver Algorithm (VQE)
  - Quantum Approximate Optimization Algorithm (QAOA)

Using the Quadratic unconstrained binary optimization (QUBO) Formulation !

Encode QUBO Into Cost Hamiltonian

$$\min f(x) = \sum_i Q_{ii} x_i + \sum_{i<j} Q_{ij} x_i x_j$$

$$f(x) = -r^T x + \lambda (c^T x - B)^2$$

$$(c^T x - B)^2 = (c^T x)^2 - 2B(c^T x) + B^2$$

$$f(x) = \sum_i (\lambda c_i^2 - 2\lambda B c_i - r_i) x_i + \sum_{i<j} (2\lambda c_i c_j) x_i x_j$$

$$x_i \to \frac{I - \sigma_i^z}{2}$$

$$H_C = \sum_i h_i \sigma_i^z + \sum_{i<j} J_{ij} \sigma_i^z \sigma_j^z$$

QAOA Algorithm Definition

- General technique that can be used to find approximate solutions to combinatorial optimization problems

- In our case, the optimization on a bitstring

QAOA Algorithm Steps

- Define a *cost Hamiltonian* such that its ground state encodes the solution to the optimization problem.
- Define a *mixer Hamiltonian*
- *Construct the Time Evolution Circuit Layers*

$$U(\boldsymbol{\gamma}, \boldsymbol{\alpha}) = e^{-i\alpha_n H_M} e^{-i\gamma_n H_C} \ldots e^{-i\alpha_1 H_M} e^{-i\gamma_1 H_C},$$

- *Prepare initial state, apply the Unitary, optimize the parameters classically*
- After the circuit has been optimized, measurements of the output state reveal approximate solutions to the optimization problem.

QAOA: Hyperparameters

- Circuit Depth: Determines the # of times U is applied.
  - We use p=1, which is fast but gives an approximate solution
  - Increasing p makes the circuit very deep, also large queue times on hardware

- The Cost Hamiltonian: Fixed by our Problem

- The Mixer Hamiltonian:
  - Engine that allows hopping between possible solutions
  - We apply X to every qubit to allow path to every other state from the initial state

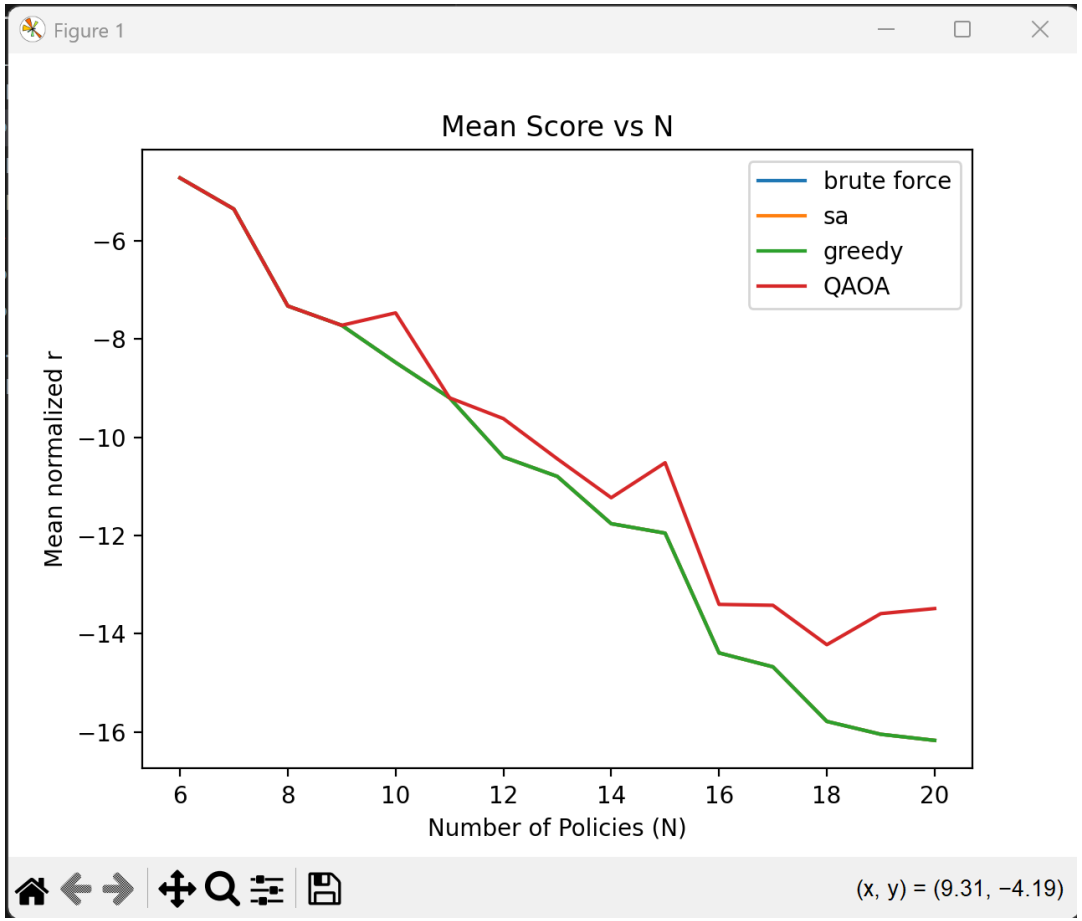


Figure 1
Mean Score vs N
brute force
sa
greedy
QAOA
Mean normalized r
−6
−8
−10
−12
−14
−16
6
8
10
12
14
16
18
20
Number of Policies (N)
(x, y) = (9.31, −4.19)

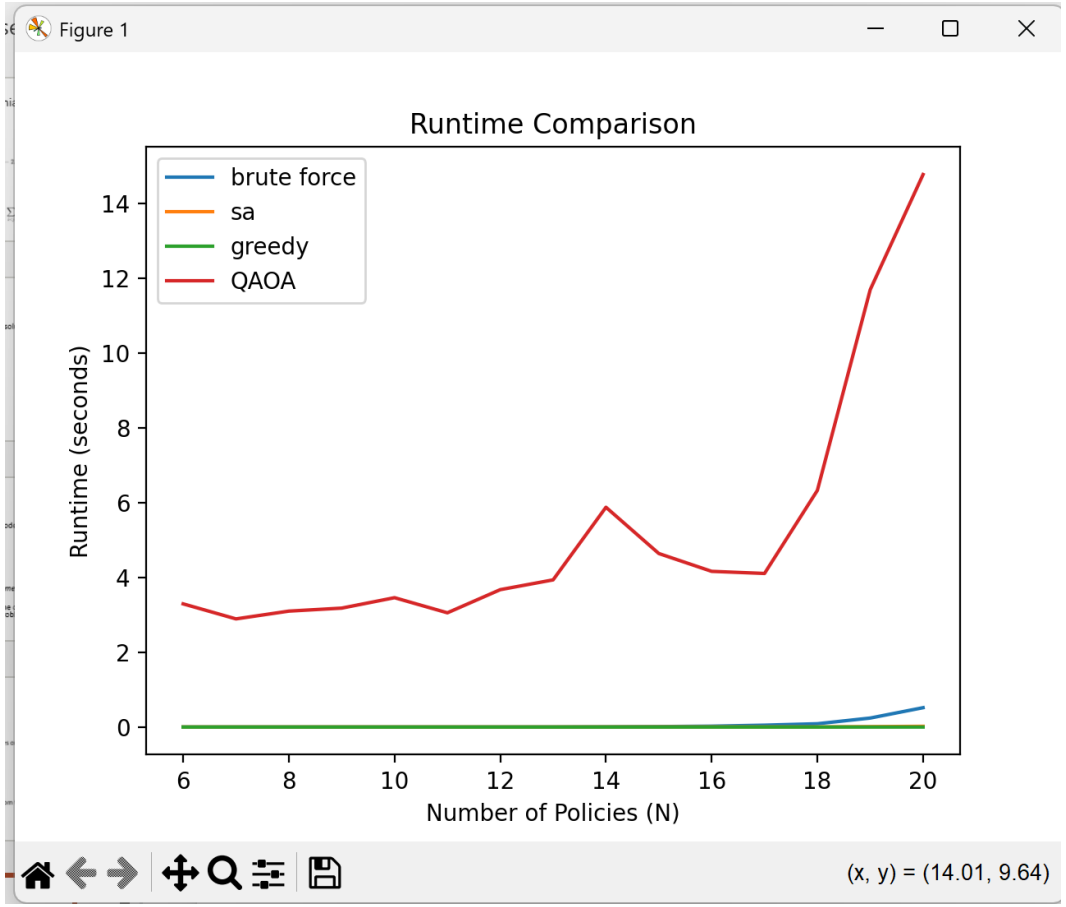


Figure 1
Runtime Comparison
brute force
sa
greedy
QAOA
Runtime (seconds)
14
12
10
8
6
4
2
0
6
8
10
12
14
16
18
20
Number of Policies (N)
(x, y) = (14.01, 9.64)